

SPARSE: Semantic Tracking and Path Analysis for Attack Investigation in Real-Time

Jie Ying , Tiantian Zhu , *Member, IEEE*, Wenrui Cheng , Qixuan Yuan , Mingjun Ma, Chunlin Xiong ,
Tieming Chen , Mingqi Lv , and Yan Chen , *Fellow, IEEE*

Abstract—As Advanced Persistent Threats (APTs) become more complex and destructive, attack investigation has gained importance. Analysts use provenance graphs for causality analysis on Point-Of-Interest (POI) events to capture critical events. However, existing methods suffer from problems of high false positives, high overhead, and high latency due to the vast size of the provenance graph and the rarity of critical events. We propose SPARSE for constructing critical component graphs (i.e., consisting of critical events) from streaming logs in real time. Our approach is based on two key observations: 1) Critical events exist in suspicious semantic graphs (SSGs) composed of interaction flows between suspicious entities, and 2) Information flows accomplishing the attacker's goal exist as paths. SPARSE uses a two-stage framework that first constructs the SSG using a state-based mode with semantic transfer rules and storage strategies. Then, it identifies suspicious flow paths (SFPs) related to the POI event and quantifies each path's influence to filter irrelevant events. Evaluation on a large-scale attack dataset shows our system generates a critical component graph (~ 113 edges) in 1.6 seconds, which is $2014 \times$ smaller than the backtracking graph ($\sim 227,589$ edges). It is also $25 \times$ more effective in filtering irrelevant edges compared to other state-of-the-art techniques.

Index Terms—Advanced persistent threat, intrusion/anomaly detection and investigation, data provenance.

Received 9 September 2024; revised 8 August 2025; accepted 11 October 2025. Date of publication 14 October 2025; date of current version 12 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2028, Grant 62372410, and Grant 62002324, in part by Zhejiang Provincial Natural Science Foundation of China under Grant LQ21F020016 and Grant LZ23F020011, and in part by The Fundamental Research Funds for the Provincial Universities of Zhejiang under Grant RF-A2023009. (Corresponding author: Tiantian Zhu.)

Jie Ying, Tiantian Zhu, Wenrui Cheng, Qixuan Yuan, Mingjun Ma, and Mingqi Lv are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: jieying@zjut.edu.cn; ttzhu@zjut.edu.cn; zjutcwr@zjut.edu.cn; zjutyqx@zjut.edu.cn; zjutmmj@zjut.edu.cn; mingqilv@zjut.edu.cn).

Chunlin Xiong is with China Unicom (Guangdong) Industrial Internet Company, Ltd., Guangzhou 510555, China (e-mail: chunlinxiong@gmail.com).

Tieming Chen is with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China, and also with the Zhejiang Key Laboratory of Visual Information Intelligent Processing, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: tmchen@zjut.edu.cn).

Yan Chen is with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL 60208 USA (e-mail: ychen@northwestern.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2025.3621434>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3621434

I. INTRODUCTION

AS THE Internet has developed over time, APT attacks have grown more sophisticated and destructive. APT attacks target mainly large corporations such as Twitter [1], resulting in significant financial losses and reputational damage. In addition, APT attacks are executed in multiple stages, which include initial access, persistence, lateral movement, collection, and exfiltration [2].

While an intrusion may be noticed at any stage, detection only uncovers isolated traces of the attack. As a result, analysts must undertake *causality analysis* to capture the bigger picture and obtain a sound understanding of the detected attack point. Achieving a secure system recovery after a cyber attack requires certain key steps. First, analysts must determine how the adversary infiltrated the system. Once the point of entry is identified, then analysts need to assess the obvious and hidden damage done to the system, such as installed payload, modified files, and exfiltrated information. In short, analysts need to identify the sequence of *critical events* leading up to the POI event and reconstruct the *critical component* (subgraph consisting of critical events), which is also called *attack investigation*.

With the improvement of kernel-level monitoring frameworks [3], [4], [5], more and more causality analysis systems depend on a provenance graph consisting of entities (e.g., files, processes, and sockets) and inter-entity interactions (e.g., processes reading and writing files). However, a provenance graph is a gigantic [6], [7] and coarse-grained data format that cannot directly determine the specific dependencies between all relevant events of an entity (e.g., a process has multiple read-in events and write-out events) [8], which leads to a dependency explosion problem and the poor performance of existing causality analysis systems [9], [10], [11].

Previous Attack Investigation Techniques and Limitations: Methodologically, causality analysis can be classified into three categories: label propagation-based, anomaly score-based, and machine learning-based. Specifically, the label propagation-based approach [12], [13], [14], [15], [16] sets entity labels and transformation rules through heuristic rules but suffers from a reliance on heavy manual effort and the incapacity to address zero-day vulnerabilities. The anomaly score-based approach [17], [18], [19], [20] quantifies the suspiciousness of dependency between entities, but faces challenges such as relying on historical statistics and the inability to adapt to complex enterprise production environments. Earlier machine learning-based

approaches [21], [22], [23], [24] attempted to learn directly from attack samples using neural networks, but were limited by insufficient sample size, poor generalization capability, and high computational overhead. Recent studies [16], [25], [26], [27], [28], [29] have employed self-supervised learning to model normal system behavior and trigger alerts through anomaly detection. However, in the field of attack investigation, these approaches can only perform basic alert correlation, such as constructing reachable paths between alerts, yet fail to reconstruct comprehensive attack scenarios. These issues make it difficult for analysts to carry out comprehensive attack investigations in the optimal time (10 minutes) [30]. In summary, a general, efficient, and cost-effective causality analysis system needs to meet the following three requirements: **1) Reduced False Positives** to address dependency explosion, **2) Affordable Overhead** to reduce the cost of attack investigation, and **3) Minimal Latency** to prevent further losses caused by subsequent attacks.

To meet the above requirements, after researching hundreds of APT attack descriptions [31] and analyzing numerous related dependency graphs [17], [19], [21], [32], [33], [34], we have the following two key observations. Firstly, *suspicious semantics are introduced externally and transferred to non-suspicious entities via interaction*. Based on this, we propose a set of predefined semantic transfer rules to process streaming logs to construct the suspicious semantic graph (SSG). The SSG consists of suspicious entities (e.g., a process visiting an unknown website) and suspicious events (data and control flows initiated by the suspicious entities) via utilizing transfer rules. We believe that the SSG contains all critical events (i.e., critical events are a subset of suspicious events) and is much smaller than the subgraph obtained by backtracking [35] from the POI event. As shown in Section V, the SSG (~ 417 edges) is $545 \times$ smaller than the backtracking graph ($\sim 227,589$ edges).

Secondly, *information flows that accomplish goals exist in the form of paths*. There are still many false positives in the suspicious semantics graph. As shown in Section V, the SSG (~ 417 edges) is 3.7 times larger than the critical component graph (~ 113 edges). Therefore, we aim to filter out the events that are contextually irrelevant to the POI event in the suspicious semantics graph by performing path-level contextual analysis. Based on this observation, we opine that evaluating whether to filter an event cannot be done in isolation, but rather calls for a comprehensive assessment of flow paths consisting of multiple events. Consequently, we construct all suspicious flow paths (SFPs) related to the POI event from SSG and quantify the degree of influence based on the properties (i.e., data flow amount and timestamp) of the POI event and path structural characteristics to weed out irrelevant events. In summary, we achieve the attack investigation through a two-stage step (i.e., SSG construction and path-level contextual analysis).

This paper proposes SPARSE¹ and makes the following contributions:

- We propose a state-based framework that contains *suspicious semantic transfer rule* and *suspicious event storage*

¹SPARSE short for Semantic tracking and Path Analysis foR attack inveStigation in real-time

TABLE I
ATTRIBUTES OF SYSTEM EVENTS

Event	Operations	Attributes
Process Event	execve, clone	Time Stamp
File Event	write, read, readv, writev	Subject Name Object Name
Network Event	sendto, recvfrom	Data Amount

strategy. The framework consumes events as streams in low overhead without recording historical data. In addition, the framework can output suspicious semantic graph related to the POI event in real time. The graph consists of all suspicious data flows and control flows that lead to the POI event, and thus contains all attack-related critical events. It is phase I of SPARSE for filtering semantic-irrelevant events.

- We propose a path-level contextual analysis mechanism that incorporates *suspicious flow path extraction and scoring*. It utilizes an optimized BFS algorithm to extract all suspicious flow paths (SFPs) from the SSG. Then the mechanism combines the properties of the POI event and characteristics of the path structure to quantify the impact of each SFP on the POI event. Finally, it filters all events that only exist in SFPs with low scores. It is phase II of SPARSE for filtering impact-irrelevant events.
- We implemented SPARSE and evaluated all its components in detail on a large-scale dataset with more than 150 million logs. Specifically, the dataset contains 10 simulated attacks [17] (~ 100 million logs) and 5 attacks from the DARPA TC program [36], [37] (~ 50 million logs). Experimental results show that SPARSE can generate the critical component graph (~ 113 edges) in 1.6s, which is $2014 \times$ smaller than the dependency graph ($\sim 227,589$ edges). The critical component graph (FP = 99) generated by SPARSE is $25 \times$ more effective than other state-of-the-art causality analysis techniques (FP = 2,473) in filtering irrelevant edges while preserving the attack sequences. In addition, SPARSE can run for a long time while processing streaming logs with a low memory overhead (30 MB).

II. BACKGROUND AND MOTIVATION

A. Dependency Graph

Recent literature has leveraged the concept of *data provenance*, i.e., instead of manually piecing together individual evidence from raw logs, provenance-based systems can construct *dependency graphs* that explain the relationships between each event, simplifying the attack investigation. Specifically, a dependency graph $G(E, V)$ is a heterogeneous graph consisting of nodes V representing system entities and edges E representing inter-entity events. The attributes of entities and events are carefully selected from raw audit logs, which are lean and critical. For entities, we choose processes ($\langle \text{ProcessName}, \text{ProcessID} \rangle$), files ($\langle \text{FileName} \rangle$), and sockets ($\langle \text{IP} : \text{Port} \rangle$). For events, we made selections as shown in Table I. For any edge $e \in E$, there is $e = (u, v, t)$, where u represents the subject, v represents the object, and t represents the timestamp of the event. For

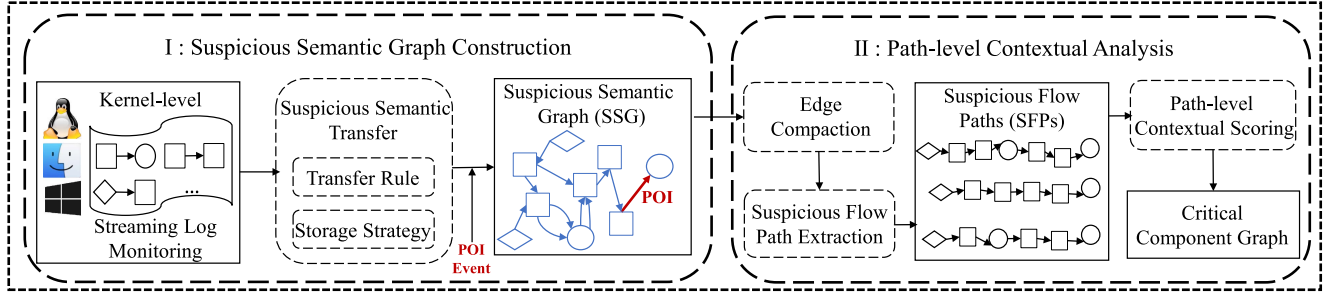


Fig. 2. Architecture of SPARSE.

Third, we exclude mimicry attacks [41] from consideration in our threat model. These attacks are designed to evade intrusion detection systems by creating a seemingly benign chain of events within an enterprise environment. Existing intrusion detection systems [42], [43], [44] often rely on heuristics or analysis of individual events, making them vulnerable to such attacks. While detecting mimicry attacks is a limitation of current detection systems, it falls outside the scope of our work. Our focus is on identifying relevant events of alert generated by the detection system as contextual information to investigate the attack.

B. Our Approach

In this section, we describe the architecture of SPARSE shown in Fig. 2. Given a POI event, SPARSE can automatically identify the critical component of the dependency graph. SPARSE consists of two phases: (I) suspicious semantic graph construction (SSGC) and (II) path-level contextual analysis (PCA).

In Phase I, SPARSE makes use of mature auditing systems [4], [45], [46], [47], [48] to access kernel-level streaming logs and process them into specific data structures. Then SPARSE proposes a *suspicious semantic transfer rule and storage strategy* to maintain the suspicious entity list and related event table with low memory overhead. Given a POI event, SPARSE can construct the *suspicious semantic graph (SSG)* in real-time.

In Phase II, SPARSE first performs edge compaction on the suspicious semantic graph. Then SPARSE proposes a *suspicious flow path extraction algorithm* to identify possible propagation paths of the data/control flow in the suspicious semantic graph (i.e., *suspicious flow paths*). Next, SPARSE performs *path-level contextual analysis*, scores each suspicious flow path, and determines how relevant the path is to the POI event. Finally, SPARSE filters out all events that only exist in irrelevant paths from suspicious semantic graph to generate the *critical component graph (CCG)* as the output.

IV. SYSTEM DESIGN

In this section, we describe the design details of each phase of SPARSE. As shown in Fig. 2, SPARSE is a two-phase framework (i.e., constructing suspicious semantic graph and performing path-level contextual analysis) for mitigating the dependency explosion problem.

A. Streaming Log Monitoring

SPARSE makes use of mature auditing systems [4], [45], [46], [47], [48] to access kernel-level logs and obtain the required data. At the entity level, SPARSE focuses on three entity types: file, process, and socket. To differentiate, SPARSE needs to construct unique identifiers for all entities. For the file, SPARSE records the absolute path as the unique identifier. For the process, SPARSE concatenates the PID and name as the unique identifier. For the socket, SPARSE constructs the 4-tuple ($\langle srcip, srcport, dstip, dstport \rangle$) as the unique identifier. At the event level, SPARSE focuses on three event types: process interactions, file IO events, and network IO events. To the best of our knowledge, existing auditing systems are rich in semantics and meet the data requirements of SPARSE.

B. Suspicious Semantic Transfer

The letter P in APT stands for persistence, which means that an attacker can lurk for a long time until achieves the goal. To support real-time investigation and long-term monitoring, SPARSE utilizes a state-based structure and suspicious semantic transfer rule to record state changes and associated events for each entity. Next, we describe the specific data structure and transfer rule in turn.

1) *Data Structure*: For any entity $v \in V$, SPARSE represents it as a triple $\langle U, T_y, S \rangle$. U is the unique identifier of the entity, the construction of U is described in Section IV-A. T_y denotes the type of the entity and S denotes the state of the entity. When S is 0 it means that the entity is not suspicious, and S is 1 it means that the entity is suspicious. Note that the file and process have their S initialized to 0 when they are created, and the socket has their S initialized to 1 when it is created, i.e., we default to all sockets that are not in the whitelist being suspicious.

For any event $e \in E$, SPARSE represents it as a quintuple $\langle U_s, U_o, O, T_i, D \rangle$. U_s and U_o are unique identifiers for the subject and object of e , respectively. O denotes the type of e , T_i denotes the time when e occurred, and D denotes the data flow amount of e . Note that SPARSE is based on the direction of the data flow and control flow to determine the location of the subject and object. For example, when $O = Read$, the data flow is from the file to the process, so the file is the subject and the process is the object. When $O = Write$, the data flow is from the process to the file, so the process is the subject and the file is the object.

TABLE II
SUSPICIOUS SEMANTIC TRANSFER RULE

Event Type	Subject	Object	Description
Recvfrom	Socket	Process	A process receives data from the network, the process becomes suspicious.
Sendto	Process	Socket	A suspicious process sends data to the network.
Read/Readv	File	Process	A process reads a suspicious file, the process becomes suspicious.
Write/Writev	Process	File	A suspicious process writes a file, the file is suspicious.
Execve/Clone	Process	Process	A process is started by a suspicious process, the process is suspicious.

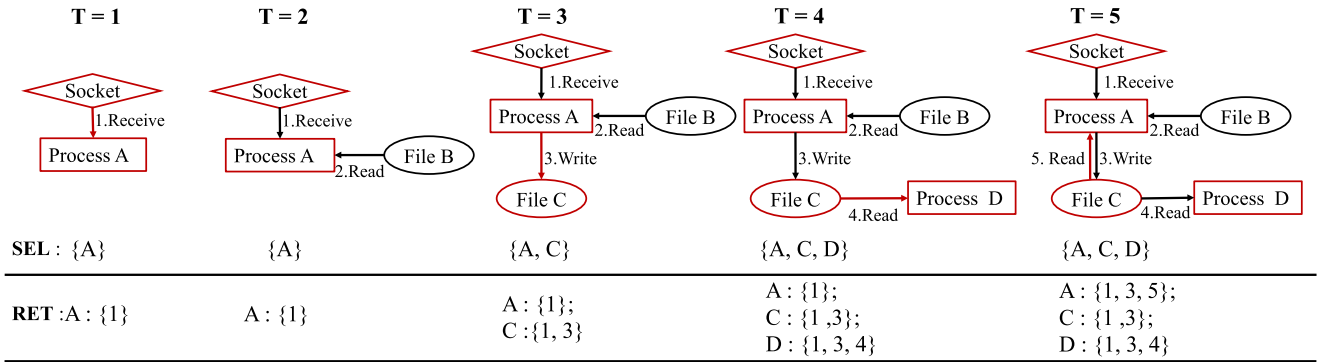


Fig. 3. An Example of Suspicious Semantic Transfer. The red solid line indicates that the entity carries suspicious semantic. **SEL** is short for Suspicious Entity List and **RET** is short for Relevant Event Table.

2) *Transfer Rule*: Based on the idea of semantic transfer, SPARSE constructs a set of predefined rules to process streaming logs and identify entity states in real-time. As shown in Table II, each rule is a quadruple: $\langle O, T_s, T_o, D \rangle$. O is the type of event, T_s and T_o are the entity types of the subject and object respectively, and D is a description of the rule. From Table II we can see that the subject can transfer suspicious semantics to the object via a specific event, which is referred to as the "suspicious semantics transfer rule". As shown in Fig. 3, T denotes the moment, red entities denote suspicious entities, and red straight arrows denote suspicious semantic transfer. When $T = 3$, a suspicious process (*process A*) writes data to a file (*file C*), which in turn carries the suspicious semantic. When $T = 4$, the suspicious file is read by another process (*process D*), which then carries the suspicious semantic. Conversely, if an entity has no suspicious semantic, any event involving this entity as a subject will not propagate suspicious semantic. For example, when $T = 2$, a file (*file B*) is read by the suspicious entity (*process A*), but there is no propagation of the suspicious semantic.

As shown in lines 5 to 11 in Algorithm 1, SPARSE processes the streaming logs, analyses data flows, and determines whether the entity state transitions. First, SPARSE accesses the event $e = \langle U_s, U_o, O, T_i, D \rangle$ and constructs the subject and object u, v corresponding to that event. Then, SPARSE determines whether the subject u is a socket or exists in *suspicious entity list* (SEL, see Section IV-B3 for detailed definition). Finally, as soon as one of these two conditions is met, SPARSE will mark the object v as suspicious and add it to SEL.

3) *STORAGE STRATEGY*: SPARSE designs two data structures to enable efficient storage of relevant data and real-time

Algorithm 1: Suspicious Semantic Graph Construction.

Input:

- 1: Streaming logs in chronological order;
- 2: Suspicious Entity List (SEL);
- 3: Related Event Table (RET);
- 4: POI event p ;

Output: Suspicious Semantic Graph (SSG) for POI event p ;

```

5: for  $e \in \text{Streaming logs}$  do
6:   Construct  $u, v$  from  $e$  where  $u_U = e_{U_s}, v_U = e_{U_o}$ 
7:   if  $u_S == 0$  and  $\nexists u_U \in \text{SEL}$  then
8:     continue;
9:   else
10:     $v_S = 1$ ;
11:    if  $\nexists v_U \in \text{SEL}$  then
12:      SEL.append( $v_U$ );
13:    end if
14:    Add  $\{v_U : \text{RET}[u_U] + e\}$  to RET;
15:  end if
16:  if  $\exists p_{U_o} \in \text{SEL}$  then
17:    return graphConstruct(RET[ $p_{U_o}$ ])
18:  end if
19: end for

```

construction of the suspicious semantic graph. Specifically, SPARSE designs a *Suspicious Entity List* (SEL) and a *Related Event Table* (RET), as defined below.

Suspicious Entity List: A list that maintains all entities with suspicious semantics (possibly related to attacks). As shown in

Algorithm 2: Suspicious Flow Path Extraction.**Input:**

- 1: (1) Suspicious Semantic Graph G ;
- 2: (2) POI Event p ;
- 3: (3) Q and V for the queue structure, T for the tree structure;

Output: Suspicious flow paths;

```

4:  $Q.add(p)$ 
5:  $T.createNode(p)$ 
6: while  $Q.num \neq 0$  do
7:    $e = Q.pop()$ 
8:    $V.add(e)$ 
9:   for  $ie \in G.inEdges(e_{U_s})$  do
10:    if  $ie_{T_i} < e_{T_i}$  and  $ie \notin Q$  and  $ie \notin V$  then
11:       $Q.add(ie)$ 
12:       $T.createNode(ie)$ 
13:       $T.createEdge(e, ie)$ 
14:    end if
15:  end for
16: end while
17: return  $T.allPaths()$ 

```

Fig. 3, when $T = 1$, the data flow passes from the suspicious socket to process A (suspicious semantic transfer), so SPARSE adds entity A to SEL. When $T = 2$, there is no suspicious semantic transfer, so SEL is not changed. When $T = 5$, the data flow passes from suspicious file C to process A , but entity A is already in SEL, so SEL is not changed.

Related Event Table: A table that holds all the related events corresponding to all suspicious entities. *The related events of a suspicious entity refer to the set of all data flows and control flows that lead to this entity's semantic change.* Specifically, SPARSE will maintain a separate set of related events in RET for all suspicious entities. Whenever an event $e = \langle U_s, U_o, O, T_i, D \rangle$ that satisfies the suspicious semantic transfer rule occurs, SPARSE will stitch the related events of U_s with event e and use it as the related events of U_o to update RET. As shown in Section V-C, the size of the RET is much smaller than the raw audit logs and the time of read in is negligible. SPARSE can construct a suspicious semantic graph related to the POI event in real-time.

As shown in Fig. 3, when $T = 1$, SPARSE adds to RET with $A : \{1\}$, indicating that the related event of the suspicious entity A is $\{1\}$. When $T = 4$, SPARSE adds to RET with $D : \{1, 3, 4\}$, stitched from the related events $\{1, 3\}$ of subject C and the current event $\{4\}$. When $T = 5$, SPARSE updates RET with $A : \{1, 3, 5\}$, stitched from the related events $\{1, 3\}$ of subject C and the current event $\{5\}$.

In order to speed up the consumption of log streams, SPARSE keeps the whole SEL in memory to determine the entity states and save suspicious entities in real-time. In contrast, inspired by the CPU architecture, SPARSE keeps only some of the high-modification (frequent growth in a short period) RETs in memory and stores other low-modification RETs in the hard disk. According to our experimental results (see Section V-C for

detail), the memory overhead of SPARSE is 30 MB on average, and there is no problem with high memory overhead. *Note that we default sockets to suspicious entities, so only entities of file type and process type are saved in SEL.*

In summary, SPARSE will use these two data structures to enable efficient storage of the necessary data and real-time construction of the suspicious semantic graph. As shown in lines 11 to 17 of Algorithm 1, SPARSE will add the object v_U to the SEL for any event $e = \langle U_s, U_o, O, T_i, D \rangle$ that satisfies the semantic transfer rule. In addition, SPARSE stitches the related events of U_s with event e and uses it as the related events of U_o to update RET. Finally, for any given POI event, SPARSE is able to extract all relevant events for the object of that POI event from the RET in real-time. SPARSE then uses a simple graph construction algorithm, which extracts entities from entities as nodes and events as edges, to construct a suspicious semantic graph associated with the POI event.

C. Edge Compaction

A suspicious semantic graph often contains multiple parallel edges between two nodes. This is because operating systems typically complete read/write tasks (e.g., file read/write) by proportionally allocating data to multiple system calls. Inspired by recent work for graph reduction [11], SPARSE merges the edges between two nodes if the time difference between them is less than a given threshold. We ultimately chose 10 seconds as it demonstrates reasonable results in terms of various system calls, such as file transfers and network connections.

D. Suspicious Flow Path Extraction

In order to perform path-level contextual analysis, it is first necessary to identify possible propagation paths of the data/control flow in the suspicious semantic graph (i.e., *suspicious flow paths*). SPARSE proposes a *suspicious flow path extraction algorithm* that can efficiently handle complex graph structures. In brief, as shown in Fig. 4, SPARSE transforms the suspicious semantic graph into a multiway tree and then traverses it to obtain all suspicious flow paths.

Specifically, as shown in lines 1 to 5 of Algorithm 2, Q and V are the queue structures, where Q holds the events to be traversed and V holds the events that have been traversed. T is the multiway tree structure, which holds the topological information. As shown in lines 6 to 9 of Algorithm 2, SPARSE traverses event e , identifying all incoming edge ies ($ies = G.inEdges(e_{U_s})$). As shown in lines 10-13 of Algorithm 2, SPARSE determines that the incoming edge ie ($ie \in ies$) occurred earlier than event e and has not been traversed ($ie \notin V$), then creates node ie in the multiway tree T and the parent of that node is e . Finally, SPARSE traverses the multiway tree T to obtain all paths from the root node to the leaf nodes, which are output as suspicious flow paths.

The suspicious flow path extraction algorithm takes into account the timeliness and directionality of the data/control flow and is able to handle the complex graph structure efficiently, as demonstrated in Section V-C, where SPARSE extracts over 140 suspicious flow paths in one second on average. Finally, it is

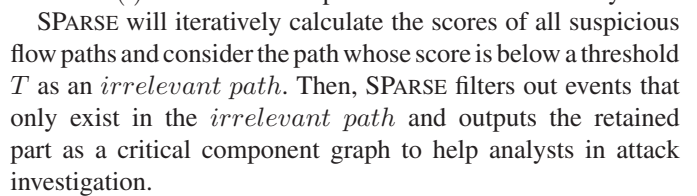


TABLE III
THE STATISTICS OF DEPENDENCY GRAPHS GENERATED FOR ALL THE 15
ATTACKS

Attack	# V	# E	# CE
Wget Executable	78	349	16
Illegal Storage	2,277	34,367	7
Illegal Storage2	9,345	290,933	7
Hide File	23,110	459,514	10
Steal Information	23,153	495,570	7
Backdoor Download	1,411	12,354	12
Annoying Server User	114	585	15
Shellshock	1,706	42,918	36
Dataleak	1,863	20,807	25
VPN Filter	2,436	39,332	29
Five Dir Case 1	259	473	8
Five Dir Case 3	6,109	83,154	9
Theia Case 1	175,196	794,341	8
Theia Case 3	281,001	1,137,829	8
Theia Case 5	245	1,309	5
Avg	35,220.20	227,589.00	13.47

V. EVALUATION

In this section, we first present the evaluation preparation, including the characteristics of the dataset, the obtaining of ground truth, and the setting of evaluation metrics. We then evaluate the effectiveness and efficiency of each component separately. In summary, we aim to answer the following questions:

- *RQ1*: How effective is SPARSE in attack investigation?
- *RQ2*: How efficient is SPARSE in attack investigation?
- *RQ3*: How sensitive is SPARSE in parameter selection?

A. Evaluation Preparation

We deploy our implementation of SPARSE on a computer with Intel (R) Core (TM) i9-10900 K CPU @ 3.70 GHz and 64 GB memory. SPARSE processes streaming logs from the auditing systems Sysdig [47] and SPADE [5], extracts information in the format as described in Section II-A, and runs continuously in a low-overhead state.

1) *Attack Dataset*: We evaluate the effectiveness of SPARSE in revealing attack sequences on a dataset with over 150 million system audit logs. As shown in Table III, this dataset contains 15 attack cases (10 simulated attacks and 5 DARPA attacks), and is provided by DEPIMPACT [17]. The simulated attacks consist of 7 (rows 2 to 8) single-host attacks based on common exploits [11], [32], [34], [49] and 3 (rows 9 to 11) multi-host attacks based on Cyber Kill Chain [50] and CVE reports [49]. The simulated attacks utilized deployed hosts with 12 active users and hundreds of processes, daily tasks such as file manipulation, text editing, and software development were carried out to simulate real-world usage. We detail these 10 simulated attacks in Appendix A1 and A2. The DARPA dataset contains 5 host attacks (rows 12 to 16), which was done by two teams (FiveDirections and Theia), and differed in terms of target systems (Windows, Linux) and vulnerability exploits (pine backdoor, firefox backdoor, and browser extension).

Table III shows the statistics of the generated dependency graphs for all attacks. Column “Attack” indicates the name of

the attack case. Columns “# V” and “# E” indicate the number of nodes and edges of the backtracking graphs after performing causality analysis [35] from POI events. Column “# CE” shows the number of critical events (related to the attack), which we explain in detail below.

2) *Obtaining Ground Truth*: In order to evaluate the performance of SPARSE, we need to specify the ground truth for all attack cases (i.e., identify all critical events). Specifically, we analyzed the targets of each attack case and determined the corresponding POI events from massive logs. We then conducted back-propagation [35] from POI events to obtain backtracking graphs and searched for critical events within them. Finally, we manually ascertained the critical events based on Indicators of Compromise (e.g., file names and malware names) and attack steps (e.g., download then execution), as shown in Appendix A.

Evaluation Metrics: First, we measure *false positives (FP)* and *false negatives (FN)*. False positives refer to those edges that SPARSE identifies as critical but are not, while false negatives refer to those edges that SPARSE identifies as irrelevant but are critical. Then we compute the false positive rate $FPR = FP/E_{total}$ and false negative rate $FNR = FN/E_c$, where E_{total} represents the number of edges and E_c represents the number of critical edges, respectively. Finally, we also computed the precision, recall, and F-score as comprehensive evaluation metrics.

B. RQ1 : How Effective is SPARSE in Attack Investigation ?

There has been a lot of graph-based related work about attack investigation [12], [13], [16], [17], [18], [23], [25], [26], [27], [29], [51], [52]. However, HOLMES [12] and RapSheet [13] rely solely on defined TTP-like (Tactics, Techniques, and Procedures) rules for detection and investigation. Such approaches suffer from heavy reliance on manual efforts and cannot effectively address zero-day vulnerabilities. HERCULE [51] and WATSON [23] address attack investigation problems by discovering communities (i.e., behavioral abstractions) on the provenance graph. Their purpose is to assist analysts in identifying attack stages from a community perspective, enabling a quick understanding of the purpose of a subgraph in the provenance graph (e.g., file compilation and uploading). HERCULE and WATSON (subgraph-level) differ in granularity from SPARSE (event-level). KAIROS [26], FLASH [25], R-CAID [29], and PROVG-SEARCHER [16] are intrusion detection systems that leverage self-supervised learning to identify anomalous behaviors and incorporate attack investigation as a subcomponent. However, these frameworks are highly integrated, with their investigation modules tightly dependent on the outputs of the detection components, making direct reuse challenging. Hence, we do not compare our work with theirs.

Here, we compare the performance of SPARSE with 4 state-of-the-art approaches: SLEUTH [52], NODOZE [18], DIST-DET [27], and DEPIMPACT [17], which are more relevant in terms of methodology (anomaly-score based) and granularity (event-level) for our evaluation. SLEUTH defines TTP-like rules with the added constraint that these rules only fire when certain confidentiality or integrity conditions are satisfied according to

TABLE IV
PERFORMANCE OF DEPENDENCY GRAPHS GENERATED BY DIFFERENT TECHNIQUES. PREC.: PRECISION; REC.: RECALL

Attack	SLEUTH			NODOZE			DISTDET			DEPIMPACT			SPARSE		
	FP	FN	# E	FP	FN	# E	FP	FN	# E	FP	FN	# E	FP	FN	# E
Wget Executable	68	7	77	78	0	94	20	0	36	32	0	48	1	0	17
Illegal Storage	2189	5	2191	5686	1	5694	1361	0	1368	2625	0	2632	47	0	54
Illegal Storage2	2072	3	2076	11959	1	11967	7738	0	7745	1255	0	1262	252	0	259
Hide File	4558	5	178703	38919	2	38931	9892	0	9902	14982	0	14992	356	0	366
Steal Information	3972	3	179202	21114	1	21122	13416	0	13423	15774	0	15781	453	0	460
Backdoor Download	1392	6	9198	232	1	245	5176	0	5188	2625	0	2637	36	0	48
Annoying Server User	2	13	281	88	2	105	29	0	44	39	0	54	1	0	16
Shellshock	672	9	4299	1007	4	1047	1847	0	1883	161	4	197	9	0	45
Dataleak	622	7	4279	597	7	629	205	0	230	44	3	69	68	0	83
VPN Filter	722	8	5342	310	5	344	482	0	511	189	2	218	72	0	101
Five Dir Case 1	97	4	237	291	1	300	271	0	25	17	0	25	3	0	11
Five Dir Case 3	277	5	37058	717	1	727	993	0	1002	171	0	180	31	0	40
Theia Case 1	453	6	264107	92091	2	92101	18719	0	18727	689	1	697	98	0	106
Theia Case 3	231	4	493626	7369	1	7378	884	0	892	1074	1	1082	61	0	129
Theia Case 5	2	1	845	395	1	401	16	0	21	3	0	8	2	0	7
Avg FP/FN/# E	1154	5.73	1163	16682	2.00	16696	4052	0	4065	2473	0.73	2487	99	0	113
Avg FPR/FNR(10^{-2})	0.507	42.54	/	7.329	14.85	/	1.78	0	/	1.087	5.419	/	0.043	0	/
Prec./Rec./F-score(%)	0.67	57.5	1.32	0.09	85.2	0.19	0.33	100	0.66	0.51	94.6	1.02	11.97	100	21.39

a tag-based information flow propagation. NODOZE measures the rarity of different events in the environment and, based on this, assigns anomaly scores to each event in the dependency graph. We use logs that only contain normal behavior (captured outside of attack periods) as execution profiles (i.e., statistics of events) to satisfy NODOZE. DISTDET is a distributed detection system that detects anomalous events by training a hierarchical system event tree (HST) model and generates alert summary graphs (ASGs) as outputs for attack investigation. Specifically, DISTDET first identifies the X-generation ancestors and Y-generation descendants of the subject process. It then incorporates the top N outgoing edges of each type (process, file, and network) from these identified nodes (i.e., the events they initiated) to form the ASG. Accordingly, we produce ASGs based on this heuristic rule and optimized hyperparameters ($X=12$, $Y=2$, $N=15$). DEPIMPACT assigns anomaly scores to edges based on several characteristics (including time, data flow amount, and node access), and then aggregates the scores to determine the entry points through a propagation algorithm. DEPIMPACT then takes as output the overlap events of the forward graph of the entry point and the backward graph of the alert point.

Table IV shows the performance of attack investigation for different techniques in all cases. Lower FP/FPR indicates a better ability to filter irrelevant edges and lower FN/FNR indicate a better ability to retain critical edges. The results show that SPARSE performs the best. On average, the critical component graph generated by SPARSE (~ 113 edges) is $8849 \times$ smaller than the original dependency graph ($\sim 1,000,000$ edges), $22 \times$ smaller than the second-best result (i.e., DEPIMPACT with $\sim 2,487$ edges). SPARSE demonstrates the best capability in filtering irrelevant edges while preserving the attack sequences ($FP = 99$, $FPR = 0.043 \times 10^{-2}$), $25 \times$ more effective than DEPIMPACT ($FP = 2,473$, $FPR = 1.087 \times 10^{-2}$). Moreover, SPARSE does not miss any critical edges (i.e., $FN = 0$, $FNR = 0$).

SLEUTH investigates attack scenarios by defining confidentiality and integrity labels for label propagation. However, SLEUTH cannot ensure to cover all the attack-related edges and therefore performed the worst in FNR ($FN = 5.73$,

$FPR = 42.54 \times 10^{-2}$), resulting in ineffective support for attack investigation. NODOZE performed better than SLEUTH in including critical edges but worse in FPR ($FP = 16,682$, $FPR = 7.329 \times 10^{-2}$; $FN = 2$, $FNR = 14.85 \times 10^{-2}$), resulting in the ineffective reduction of investigation cost for analysts. The reason is that the performance of NODOZE relies on whether the execution profile comprehensively covers all benign behaviors but ignores information in the form of streams. DISTDET considers the ancestor process nodes up to X preceding generations ($X = 12$), the descendant process nodes up to Y subsequent generations ($Y = 2$), and the top N ($N = 15$) outgoing nodes as part of the ASG. This comprehensive inclusion enables DISTDET to achieve optimal performance in terms of false negatives ($FN = 0$). However, due to the lack of further filtering mechanisms to eliminate irrelevant events, DISTDET performs poorly in false positive reduction ($FP = 4052$, $FPR = 1.72 \times 10^{-2}$). DEPIMPACT heuristically selects 3 entry points (one each for files, processes, and sockets), which amplifies the attack surface. DEPIMPACT directly takes the intersection between the forward graph of the entry points and the backward graph of the alert point as output, which introduces massive attack-irrelevant events ($FP = 2,473$, $FPR = 1.087 \times 10^{-2}$). Regarding the reduction results, SPARSE exhibits the best performance ($FP = 99$, $FPR = 0.043 \times 10^{-2}$), we attribute the good performance of our system to (1) SPARSE employs semantic transfer to construct a suspicious semantic graph related to the POI event that inherently filters out a significant number of irrelevant events ($FP = 403$, $FPR = 0.177 \times 10^{-2}$), and (2) SPARSE evaluates the relevance of context to POI events at the path-level rather than in isolation.

Finally, SPARSE outperforms all comparative methods across all comprehensive evaluation metrics (i.e., precision, recall, and F-score), indicating that it effectively meets the practical requirement for low false positives.

C. RQ2 : How Efficient is SPARSE in Attack Investigation ?

In this section, we evaluate the efficiency of SPARSE when deployed in a real scenario. First, we evaluate SPARSE on the real-time performance by comparing data generation rate and

TABLE V
COMPARISON OF DATA CONSUMPTION RATE AND DATA GENERATION RATE

Attack Case	Generation Rate (event num/pers)	Consumption Rate (event num/pers)
Wget Executable	3,018	42,175
Illegal Storage	3,273	45,572
Illegal Storage2	3,156	34,727
Hide File	4,039	31,155
Steal Information	4,271	40,719
Backdoor Dowanload	3,380	39,451
Annoying Server User	3,153	42,155
Shellshock	3,741	42,742
Dataleak	3,692	42,899
VPN Filter	3,560	39,611
Five Dir Case 1	917	40,358
Five Dir Case 3	1,161	44,795
Theia Case 1	1,298	32,097
Theia Case 3	1,215	34,856
Theia Case 5	966	41,303
Avg	2,722	39,641

data consumption rate. Table V shows that SPARSE can consume events $15 \times$ faster than the events generation rate of the host on average, which shows the real-time of SPARSE is feasible. Then we evaluate the overheads of each component of SPARSE on time, memory, CPU, and disk, as shown in Table VI. In addition, we conduct comparative experiments with the baseline method DEPIMPACT.

Time and Memory: SPARSE can perform path-level contextual analysis of the suspicious semantic graph and construct the critical component graph in 2 s on average, which is $4091 \times$ faster than DEPIMPACT ($\sim 6,464$ s). DEPIMPACT uses an impact propagation algorithm to identify entry points. However, when the number of edges in the backtracking graph is high (e.g., case “Hide File”), it takes an extremely long time to reach global convergence (19,814s). In terms of memory overhead, three causes are making SPARSE smaller than DEPIMPACT: (1) SPARSE only stores suspicious nodes in memory when processing streaming logs, so the memory overhead grows extremely slowly and can be seen as constant in scale (i.e., 30 MB). (2) The suspicious semantic graphs (~ 403 edges) read in by SPARSE are much smaller than the dependency graphs ($\sim$ one million edges) read in by DEPIMPACT. (3) The suspicious flow path extraction algorithm applied in PCA only traverses the graph structure once (a complexity of $O(E)$) and generates a smaller number of SFPs (230 on average). For these reasons, the memory overhead of PCA (132.82 MB) is $2.4 \times$ smaller than that of DEPIMPACT (320.81 MB). Finally, it is important to emphasize that SPARSE is real-time for suspicious semantic graph construction (SSGC), so we do not perform a time overhead evaluation for this component.

CPU and Disk: SPARSE stores the relevant event table (RET) in the database when processing streaming logs. To evaluate the overhead of SPARSE on the hard disk, we also perform the relevant experiments. As shown in Table VI, SPARSE requires only 21.03 MB of disk space, which is $9 \times$ smaller than the original logs (204.87 MB). This is because SPARSE only retains events of suspicious semantic relevance, whereas other techniques [10], [17], [18], [19], [34] require the whole logs, which significantly

increases overhead on the disk. In addition, the CPU overhead of SPARSE is 4.5%, due to the simple yet intuitive and effective algorithm.

In summary, we believe that SPARSE outperforms the latest work DEPIMPACT in all aspects of overhead and can satisfy the requirements of low overhead and latency in application scenarios.

D. RQ3: How Sensitive is SPARSE in Parameter Selection?

As shown in Section IV-E, there are two super parameters in SPARSE that need to be set: (1) the C affects the expansion factor, (2) the T filters the paths. As shown in Equation (3), the larger C , the smaller the expansion factor α , the lower the path score, and the fewer events will be retained by SPARSE. Parameter T , on the other hand, indicates the severity of SPARSE for path selection; the larger T is, the fewer events will be retained by SPARSE.

We demonstrate the sensitivity of SPARSE in parameter selection by testing all combinations of the hyper-parameters C and T through grid search methodology. Specifically, we set the minimum value of C to be 1, the maximum value to be 9, and the step size to be 1; set the minimum value of T to be 0.1, the maximum value to be 0.9, and the step size to be 0.1; and test the performance of SPARSE on the metrics FP/FN as shown in Fig. 5. As parameter C increases, SPARSE reduces the path score and retains fewer events, hence the FP decreases. At the same time, SPARSE misses some critical events, causing FN to rise. As the threshold T increases, SPARSE blocks more paths and preserves fewer events, so FP falls while FN rises. As shown in Fig. 5(a), FP decreases gradually from the upper left to the lower right; as shown in Fig. 5(b), FN increases gradually from the upper left to the lower right. The effects of these parameter changes on the SPARSE are in line with our predictions.

Obviously, FP and FN are two evaluation metrics that we both want to be as low as possible, but there is a trade-off between their performance for a system (i.e., a rise in one leads to a fall in the other). Finally, we choose $C = 5$ and $T = 0.5$ as the default parameters for SPARSE. Of course, the manufacturer can adapt these parameters to the specific scenario.

VI. DISCUSSION

Cooperation with existing techniques: There is a requirement for defenders to be able to detect and handle real-world attacks in real time. As an attack investigation system, SPARSE can be combined with a variety of existing techniques to meet this goal. By working with intrusion detection systems [12], [14], [15], [53] that can provide real-time alerts and defenses, SPARSE is able to investigate alerts for relevant events and provide a brief critical component graph to analysts. By working with compression systems [10], [11], [38], [54], [55] that can reduce redundant information, SPARSE is able to reduce memory and disk overheads, enabling years of relevant data storage. By working with analysis systems [13], [18], [56] that automatically determine the authenticity of alarms, SPARSE is able to provide streamlined but sufficient relevant events to support the triage of alarms.

TABLE VI
OVERHEAD PERFORMANCE OF EACH COMPONENT OF SPARSE AND BASELINE APPROACH

Attack	SSGC			PCA				DEPIMPACT			
	Mem. (MB)	CPU (%)	Disk (MB)	Ti. (s)	Mem. (MB)	CPU (%)	# SFP	Ti. (s)	Mem. (MB)	CPU (%)	Disk (MB)
Wget Executable	29.75	3.14	0.072	0.09	108.20	4.97	5	4.45	125.72	7.62	0.79
Illegal Storage	29.92	3.32	7.15	0.32	124.97	4.36	85	285.10	178.89	7.46	79.21
Illegal Storage2	31.81	3.35	69.21	2.38	125.16	4.27	522	985.55	304.51	7.29	169.21
Hide File	34.82	3.59	101.12	6.22	147.34	4.94	833	19,814.25	592.82	7.51	401.12
Steal Information	36.55	3.61	109.17	6.66	155.60	4.90	868	12,701.39	596.26	7.82	409.17
Backdoor Dowanload	28.01	3.27	2.21	0.10	125.16	4.49	20	33.68	199.17	7.42	61.13
Annoying Server User	26.62	3.40	0.011	0.33	119.41	4.41	6	0.23	127.81	7.76	0.70
Shellshock	29.74	3.39	8.94	0.02	137.75	4.13	40	2.17	138.98	7.57	154.00
Dataleak	30.39	3.40	0.82	0.04	132.86	4.29	17	2.24	130.65	7.22	97.50
VPN Filter	31.01	3.26	0.82	0.06	146.08	4.27	37	15.46	135.49	7.43	537.95
Five Dir Case 1	28.83	3.29	7.15	0.04	124.28	4.56	11	2.08	124.12	7.67	0.82
Five Dir Case 3	30.05	3.50	37.71	0.11	135.04	4.29	22	1,674.90	189.08	7.72	88.65
Theia Case 1	31.97	3.43	59.50	2.34	137.35	4.15	374	24,761.90	844.28	7.64	479.20
Theia Case 3	35.08	3.62	68.45	4.85	143.90	4.80	598	36,682.87	885.83	7.79	592.23
Theia Case 5	29.12	3.30	0.13	0.19	129.21	4.67	15	4.91	138.60	7.15	1.41
Avg	30.91	3.39	21.03	1.58	132.82	4.50	230	6464.75	320.81	7.54	204.87

† SSGC: Suspicious Semantic Graph Construction. PCA: Path-level Contextual Analysis. SFP: Suspicious Flow Path.

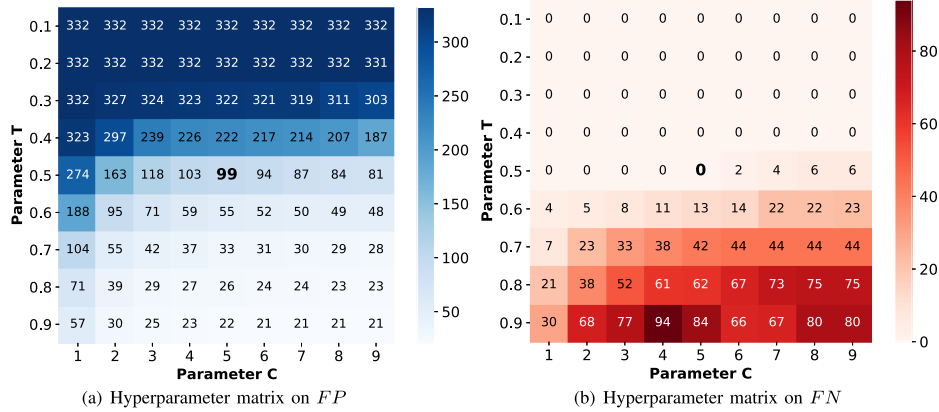


Fig. 5. Hyperparameter Matrices for System Performance with Different Parameters.

Evasion Attacks: Existing investigation techniques, such as DEPIMAPCT, utilize weight computation and score-propagation techniques to identify attack entry points. However, this insight of independently calculating the weights of events does not fit the situation where information flows between entities. As a result, an attacker can inject a payload by writing multiple times and thus evade tracking. In contrast, SPARSE mitigates the impact of this by performing contextual analysis in path-level to synthesize the relevance between a path and an alert. An attacker may evade investigation by going the long way around (i.e., repeating nonsensical behavior) as in the attack case “Hide File” where the attacker changes the file name multiple times. As shown in Section IV-E, SPARSE is able to mitigate this problem by performing relative score calculation and event score inflation mechanisms.

Limitation: To implement attack investigation, SPARSE relies on alerts initiated by Endpoint Detection and Response (EDR) placed on the host. SPARSE cannot perform attack investigation if the detection system fails to launch alerts (identify suspicious behavior). Recent approaches [57], [58] propose solutions to

improve the detection of anomalous system activity, and SPARSE can work with these approaches to provide better defenses. If the detection system initiates false positives frequently, SPARSE can only identify relevant events but cannot filter these false alerts. But SPARSE can work with alarm triage techniques [13], [18], [54] to help them filter false alarms by providing a streamlined critical component graph. In addition, as shown in Section IV-B3, SPARSE needs to maintain a suspicious entity list in memory and a related event table on disk. As the runtime grows, there is redundant information in the related event table, such as *Event 10* being stored four times in Fig. 3. By working with existing compression systems [10], [11], [38], [54], [55], SPARSE can effectively mitigate this situation and enable long deployment runs.

VII. RELATED WORK

The analysts need to perform threat alert validation and post-mortem analysis of incidents. Currently, while auditing is by no means the only form of forensic investigation, it is telling

that 75% of incident response specialists consider logs to be the most valuable form of investigation artifact [59]. And in terms of methodologies for attack investigation using logs, they can be categorized into three categories: label propagation-based, anomaly score-based, and machine learning-based.

Label Propagation-based: Labels are given to nodes and are propagated to other nodes by system calls. When an alert arises, the analyst can easily retrace the events associated with the alert based on the label. Milajerdi et al. propose HOLMES [12], which mitigates the dependency explosion problem by requiring the aggregation of more labels to raise the detection threshold. RapSheet [13] makes use of the tactical provenance graph (TPG), which instead of encoding low-level system event dependencies, reasons about the causal relationships between threat alerts. RapSheet proposes a threat scoring scheme that evaluates the severity of each alert based on TPGs to enable effective investigation of alerts. CONAN [14] iteratively performs malicious behavior determination with label passing and aggregation through data provided by ETW, enabling real-time detection and investigation. ProvG-Searcher [16] employs graph reduction techniques to address the semantic gap between the query graph and the provenance graph and simplify subgraph matching to straightforward comparisons between a query and precomputed subgraph representations by using order embeddings. These label-based approaches rely on heuristic rules that cannot handle all types of attacks and have a high level of false negatives in attack investigation.

Anomaly Score-based: The essential view of these methods is to quantify the suspiciousness of edges between pairs of nodes. Pei et al. 's HERCULE [51] system correlates multi-source heterogeneous logs to construct a multi-dimensional weighted graph and uses the unsupervised community detection algorithm Louvain [60] to discover attack-related paths from it. NODOZE [18] and PRIOTRACKER [19], on the other hand, perform statistics on historical data and assign anomaly scores to events in the dependency graph. The score propagation algorithm is then used to find suspicious events. Both of these methods rely on statistics of historical data and cannot be applied in complex and variable generative environments. DEPIMPACT [17] calculates dependency weights globally based on multiple features (including time, data traffic amount, and node access) and then aggregates the weights to nodes to determine suspicious points of intrusion. The overlapping parts between the forward graph of the entry point and the backward graph of the alarm point are then considered attack-related events. These score-based methods can cover all critical events. However, there is no restriction on the variation of scores to solve the dependency explosion problem, thus leading to higher false positives. In addition, these methods require reading in all the logs to build the dependency graph, which is very expensive in terms of hard disk and memory.

Machine Learning-based: Some techniques use machine learning methods to learn contextual and structural information from dependency graphs to identify the most relevant abnormal events to alert. ATLAS [21] uses a novel combination of causal analysis, natural language processing, and machine learning to construct sequence-based models as a way to

establish critical patterns of attack and non-attack behavior in the dependency graph. On the other hand, DEPCOMM [22] proposes a novel graph summarization method by dividing the large graph into process-centric subgraphs. DEPCOMM then extracts summaries from each subgraph, enabling the generation of summary graphs from dependency graphs, thereby reducing the difficulty of investigation for analysts. ConGraph [24] integrates provenance graphs with multi-process contextual information and employs a CNN-BiLSTM model to detect anomalous nodes. These supervised learning-based approaches suffer from inadequate training samples, poor generalization capabilities, and high computational costs. Therefore, some studies leverage self-supervised learning to model normal system behavior, enabling the generation of alerts upon detecting anomalies. These alerts are subsequently correlated to produce the final output for attack investigation. DISTDET [27] is a distributed detection system that detects anomalous events by training a hierarchical system event tree (HST) model. Then, DISTDET marks detected anomalous events as alerts and generates alarm summary graphs (ASGs). KAIROS [26] trains an anomaly detection model with an encoder and decoder architecture and reconstructs the attack through community discovery algorithms [60] to help security analysts quickly understand the full attack scene. FLASH [25] incorporates a novel adaptation of a GNN-based contextual encoder to efficiently encode both local and global graph structures into expressive node embeddings. Similarly, Flash implements attack investigation at the end through the construction of an attack evolution graph. MAGIC [28] uses masked graph representation learning to model benign system entities and behaviors, but without an explicit attack investigation component. R-CAID [29] precomputes the root cause of each node during the graph construction phase and directly links each node to its root during the embedding process, thereby preventing adversaries from masquerading as legitimate processes. Finally, R-CAID includes all root cause nodes of the anomalous nodes as part of the attack investigation output. However, the investigation components in the aforementioned works typically rely on simple correlation of multiple alerts and exhibit a high degree of integration, making them heavily dependent on the outputs of preceding detection components.

In contrast to previous work, SPARSE employs a hybrid method in the specific domain of causality tracking. SPARSE first uses *suspicious semantic delivery rule* to construct suspicious semantic graph. Then SPARSE uses *path-level contextual analysis* to extract a streamlined critical component graph.

VIII. CONCLUSION

We propose SPARSE, a system that processes streaming logs and outputs critical events (attack-related events) according to alert in real-time. Specifically, SPARSE constructs a suspicious semantic graph related to the POI event by *suspicious semantic transfer rule and storage strategy*. Then SPARSE uses a *suspicious flow path extraction algorithm* to extract all reachable flow paths from the suspicious semantic graph. Finally, SPARSE uses *path-level contextual analysis* to score all paths and filters irrelevant events to obtain the final critical component graph.

Our evaluation of real attacks demonstrates that SPARSE achieves low false positives (FP = 99), low overhead (30 MB for memory and 21.03 MB for hard disk), and low latency (1.58 s for attack investigation).

REFERENCES

- [1] “What Twitter’s 200 million-user email leak actually means,” 2024. [Online]. Available: <https://www.wired.com/story/twitter-leak-200-million-user-email-addresses/>
- [2] “Mitre att&ck,” 2024. [Online]. Available: <https://attack.mitre.org/>
- [3] “System administration utilities,” 2024. [Online]. Available: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/6/html/security_guide/chap-system_auditing
- [4] “About event tracing,” 2024. [Online]. Available: <https://docs.microsoft.com/en-us/windows/win32/etw/about-event-tracing>
- [5] A. Gehani and D. Tariq, “Spade: Support for Provenance Auditing in Distributed Environments,” in *Proc. ACM/IFIP/USENIX Int. Middleware Conf.*, 2012, pp. 101–120.
- [6] S. Ma et al., “Kernel-supported cost-effective audit logging for causality tracking,” in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, 2018, pp. 241–254.
- [7] A. Bates et al., “Trustworthy whole-system provenance for the Linux kernel,” in *Proc. 24th USENIX Conf. Secur. Symp.*, 2015, pp. 319–334.
- [8] M. A. Inam et al., “SoK: History is a vast early warning system: Auditing the provenance of system intrusions,” in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 307–325.
- [9] K. H. Lee et al., “High accuracy attack provenance via binary-based execution partition,” in *Proc. Netw. Distrib. Syst. Secur.*, 2013, vol. 16, pp. 1–16.
- [10] Y. Tang et al., “NodeMerge: Template based efficient data reduction for big-data causality analysis,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1324–1337.
- [11] Z. Xu et al., “High fidelity data reduction for Big Data security dependency analyses,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 504–516.
- [12] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-time APT detection through correlation of suspicious information flows,” in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.
- [13] W. U. Hassan, A. Bates, and D. Marino, “Tactical provenance analysis for endpoint detection and response systems,” in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1172–1189.
- [14] C. Xiong et al., “Conan: A practical real-time APT detection system with high accuracy and efficiency,” *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 551–565, Jan./Feb. 2020.
- [15] T. Zhu et al., “APTSIELD: A stable, efficient and real-time APT detection system for linux hosts,” *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5247–5264, Nov–Dec. 2023.
- [16] E. Altinisik, F. Deniz, and H. T. Sencar, “ProvG-searcher: A graph representation learning approach for efficient provenance graph search,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2247–2261.
- [17] P. Fang et al., “Back-propagating system dependency impact for attack investigation,” in *Proc. 31th USENIX Secur. Symp.*, 2022, pp. 2461–2478.
- [18] W. U. Hassan et al., “Nodoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. Netw. Distrib. Syst. Secur.*, 2019, pp. 1–15.
- [19] Y. Liu et al., “Towards a timely causality analysis for enterprise security,” in *Proc. Netw. Distrib. Syst. Secur.*, 2018, pp. 1–15.
- [20] Hassan et al., “This is why we can’t cache nice things: Lightning-fast threat hunting using suspicion-based hierarchical storage,” in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 165–178.
- [21] A. Alsaheel et al., “ATLAS: A sequence-based learning approach for attack investigation,” in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3005–3022.
- [22] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, “DEPCOMM: Graph summarization on system audit logs for attack investigation,” in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 540–557.
- [23] J. Zeng et al., “WATSON: Abstracting behaviors from audit logs via aggregation of contextual semantics,” in *Proc. Netw. Distrib. Syst. Secur.*, 2021, pp. 1–18.
- [24] L. Li and W. Chen, “Congraph: Advanced persistent threat detection method based on provenance graph combined with process context in cyber-physical system environment,” *Electronics*, vol. 13, no. 5, 2024, Art. no. 945.
- [25] M. U. Rehman, H. Ahmadi, and W. U. Hassan, “Flash: A comprehensive approach to intrusion detection via provenance graph representation learning,” in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3552–3570.
- [26] Z. Cheng et al., “KAIRIS: Practical intrusion detection and investigation using whole-system provenance,” in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3533–3551.
- [27] F. Dong et al., “{DISTDET}: A { Cost-Effective} distributed cyber threat detection system,” in *Proc. 32nd USENIX Secur. Symp.*, 2023, vol. 23, pp. 6575–6592.
- [28] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, and M. Wen, “{MAGIC}: Detecting advanced persistent threats via masked graph representation learning,” in *Proc. 33rd USENIX Secur. Symp.*, 2024, vol. 24, pp. 5197–5214.
- [29] A. Goyal, G. Wang, and A. Bates, “R-CAID: Embedding root cause analysis within provenance-based intrusion detection,” in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3515–3532.
- [30] “Lateral movement,” 2024. [Online]. Available: <https://www.crowdstrike.com/cybersecurity-101/lateral-movement/>
- [31] “Apt notes,” 2024. [Online]. Available: <https://github.com/aptnotes/data/>
- [32] Y. Kwon et al., “MCI: Modeling-based causality inference in audit logging for attack investigation,” in *Proc. Netw. Distrib. Syst. Secur.*, 2018, vol. 2, pp. 1–14.
- [33] P. Gao et al., “{AIQL}: Enabling efficient attack investigation from system monitoring data,” in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, 2018, pp. 113–126.
- [34] S. Ma et al., “ProTracer: Towards practical provenance tracing by alternating between logging and tainting,” in *Proc. Netw. Distrib. Syst. Secur.*, 2016, vol. 2, pp. 1–15.
- [35] S. T. King and P. M. Chen, “Backtracking intrusions,” *ACM SIGOPS Operating Syst. Rev.*, vol. 37, no. 5, pp. 223–236, 2003.
- [36] “Darpa,” 2024. [Online]. Available: <https://www.darpa.mil/program/transparent-computing>
- [37] “Darp3 transparent engagement 3,” 2023. [Online]. Available: <https://drive.google.com/drive/folders/1QlbUFWAGq3Hpl8wVdzOdIoZLFxkII4EK>
- [38] T. Zhu et al., “General, efficient, and real-time data compaction strategy for APT forensic analysis,” *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3312–3325, 2021.
- [39] P. Gao et al., “Enabling efficient cyber threat hunting with cyber threat intelligence,” in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 193–204.
- [40] Gao et al., “{SAQL}: A stream-based query system for real-time abnormal system behavior detection,” in *Proc. 27th USENIX Conf. Secur. Symp.*, 2018, pp. 639–656.
- [41] D. Wagner and P. Soto, “Mimicry attacks on host-based intrusion detection systems,” in *Proc. 9th ACM Conf. Comput. Commun. Secur.*, 2002, pp. 255–264.
- [42] M. Bishop et al., *Introduction to Computer Security*, vol. 50. Reading, MA, USA: Addison-Wesley, 2005.
- [43] C. Kruegel et al., *Intrusion Detection and Correlation: Challenges and Solutions*, vol. 14. Berlin, Germany: Springer, 2004.
- [44] “Insider threat monitoring software,” 2023. [Online]. Available: https://www.netwrix.com/insider_threat_detection.html
- [45] “Auditd,” 2023. [Online]. Available: <https://linux.die.net/man/8/auditd>
- [46] “Lttng,” 2023. [Online]. Available: <https://lttng.org>
- [47] “Sysdig,” 2023. [Online]. Available: <https://github.com/draios/sysdig>
- [48] “Redhat,” 2023. [Online]. Available: <https://github.com/linux-audit/>
- [49] “Exploit database,” 2024. [Online]. Available: <https://www.exploit-db.com/>
- [50] “Cyber kill chain,” 2023. [Online]. Available: <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>
- [51] K. Pei et al., “HERCULE: Attack story reconstruction via community discovery on correlated log graph,” in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 583–595.
- [52] M. N. Hossain et al., “SLEUTH: Real-time attack scenario reconstruction from cots audit data,” in *Proc. 26th USENIX Conf. Secur. Symp.*, 2017, pp. 487–504.
- [53] T. Kim et al., “Intrusion recovery using selective re-execution,” in *Proc. 9th USENIX Conf. Operating Syst. Des. Implementation*, 2010, pp. 89–104.
- [54] M. N. Hossain et al., “Dependence-preserving data compaction for scalable forensic analysis,” in *Proc. 27th USENIX Conf. Secur. Symp.*, 2018, pp. 1723–1740.
- [55] N. Michael et al., “On the forensic validity of approximated audit logs,” in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 189–202.

- [56] W. U. Hassan et al., "This is why we can't cache nice things: Lightning-fast threat hunting using suspicion-based hierarchical storage," in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 165–178.
- [57] S. Wang et al., "Heterogeneous graph matching networks for unknown malware detection," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 3762–3770.
- [58] X. Han et al., "Unicorn: Runtime provenance-based detector for advanced persistent threats," *Netw. Distrib. Syst. Secur. Symp.*, pp. 1–18, 2020.
- [59] "Carbon black," 2018. [Online]. Available: <https://www.carbonblack.com/global-incident-response-threatreport/november-2018/>
- [60] V. D. Blondel et al., "Fast unfolding of communities in large networks," *J. Stat. Mech.: Theory Experiment*, vol. 2008, no. 10, 2008, Art. no. P10008.
- [61] Vpnfilter, New router malware with destructive capabilities," 2018. [Online]. Available: <https://symc.ly/2IPGGVE>
- [62] Ebay, "Ebay Inc. to ask Ebay users to change pass-words" 2014. [Online]. Available: <http://blog.ebay.com/ebay-inc-ask-ebay-users-change-passwords/>
- [63] "Schneier security: Router vulnerability the vpnfilter botnet," 2018. [Online]. Available: https://www.schneier.com/blog/archives/2018/06/router_vulnerab.html



Mingjun Ma received the BE degree in computer science from Zhejiang ShuRen University, HangZhou, China, in 2023. He is currently working toward the MD degree in computer science with the Zhejiang University of Technology. His research interests include APT and cyber intelligence and mobile authentication.



Chunlin Xiong received the PhD degree on cybersecurity from Zhejiang University, Hangzhou, China, in 2021. He is currently the chief security expert of China Unicom (Guangdong) Industrial Internet Company, Ltd., responsible for the construction of SOC and large models in the security field.



Jie Ying received the BE degree in digital media from the Zhejiang University of Technology, Hangzhou, China, in 2020. He is currently working toward the PhD degree in computer science the Zhejiang University of Technology. His research interests include APT and cyber threat Intelligence.



Tiantian Zhu (Member, IEEE) received the PhD degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an associate professor with the College of Computer Science and Technology, Zhejiang University of Technology, China. His research interests include system security and artificial intelligence.



Tieming Chen received the PhD degree in computer software and theory from BeiHang University, Beijing, China, in 2011. He is currently a professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou, China. His research interests include cyberspace security and intelligence security. He is also a member of ACM.



Wenrui Cheng received the BE degree in computer science and technology from Huzhou University, Huzhou, China, in 2022. She is currently working toward the PhD degree in computer science with Zhejiang University of Technology. Her current research interests include knowledge driven APT attack sample generation and detection.



Mingqi Lv received the PhD degree in computer science from Zhejiang University, Hangzhou, China, in 2012. His research interests include spatiotemporal data mining and ubiquitous computing.



Qixuan Yuan received the BE degree in computer science and technology from ShanDong Normal University, Jinan, China, in 2022. She is currently working toward the PhD degree in computer science with Zhejiang University of Technology. Her research interests include APT detection and cyber threat intelligence analysis.



Yan Chen (Fellow, IEEE) received the PhD degree in computer science from the University of California, Berkeley, CA, USA, in 2003. He is currently a professor with the Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL, USA. His research interests include network security, measurement, and diagnosis for large-scale networks and distributed systems.